

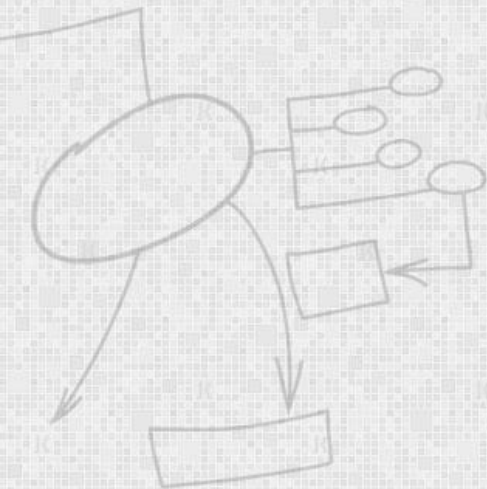
EC-360

SOFTWARE ENGINEERING

Lecture 6

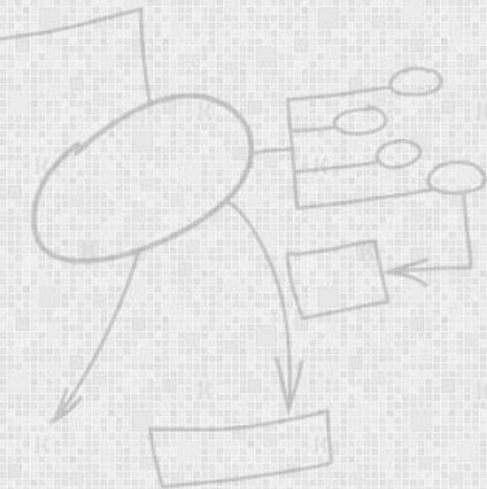
Software Design Patterns

By
Dr Wasi Haider Butt



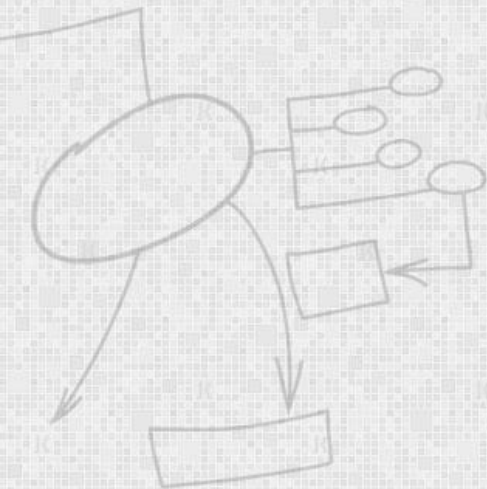
Agenda

- Intro to Design Patterns
- GoF Design Patterns Classification
- Singleton Design Pattern
- Façade Design Pattern



Patterns

“Each pattern describes a design *problem* which occurs *over and over again* in our environment, and then describes the core of the *solution* to that problem, in such a way that you can use this solution again and again.”



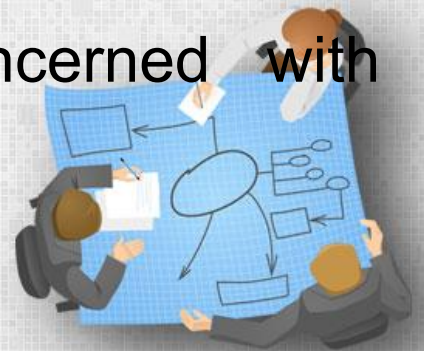
“Gang of Four” (GoF) Book

- Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley Publishing Company
- Written by this "gang of four"
 - Dr. Erich Gamma, then Software Engineer, Taligent, Inc.
 - Dr. Richard Helm, then Senior Technology Consultant, DMR Group
 - Dr. Ralph Johnson, then and now at University of Illinois, Computer Science Department
 - Dr. John Vlissides, then a researcher at IBM
 - Thomas J. Watson Research Center
 - See John's WikiWiki tribute page <http://c2.com/cgi/wiki?JohnVlissides>



Object-Oriented Design Patterns

- This book defined 23 patterns in three categories
 - **Creational patterns** deal with the process of object creation instead of using “new” for object creation
 - **Structural patterns:** deal with class and object composition. The concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionality
 - **Behavioral patterns,** are specifically concerned with communication between objects.



GoF Patterns

– *Creational Patterns*

- Abstract Factory
- Builder
- Factory Method
- Prototype
- Singleton

– *Structural Patterns*

- Adapter
- Bridge
- Composite
- Decorator
- Façade
- Flyweight
- Proxy

– *Behavioral Patterns*

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor



Style for Describing Patterns

- General structure:
 - *Pattern name*
 - *Recurring problem*: what problem the pattern addresses
 - *Solution*: the general approach of the pattern
 - *UML for the pattern*
 - *Participants*: a description as a class diagram
 - *Use Example(s)*: examples of this pattern, in code



Singleton Pattern

- The Singleton pattern ensures that a class has only one instance and provides a global point of access to it.
- Examples:
 - There can be many printers in a system but there should only be one printer spooler.
 - There should be only one instance of a WindowManager.
 - There should be only one instance of a filesystem.



Singleton Pattern

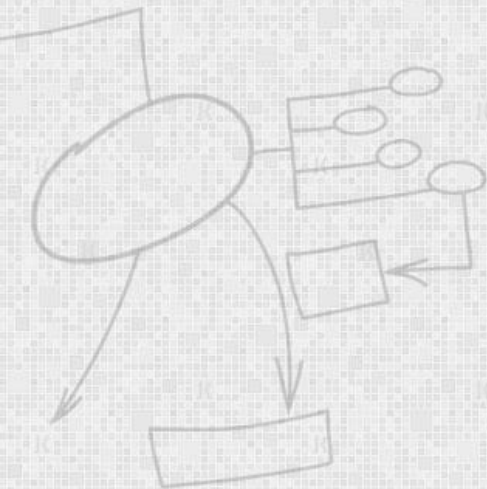
- How do we ensure that a class has only one instance and that the instance is easily accessible?
- A global variable makes an object accessible, but does not keep you from instantiating multiple objects.
- A better solution is to make the class itself responsible for keeping track of its sole instance.



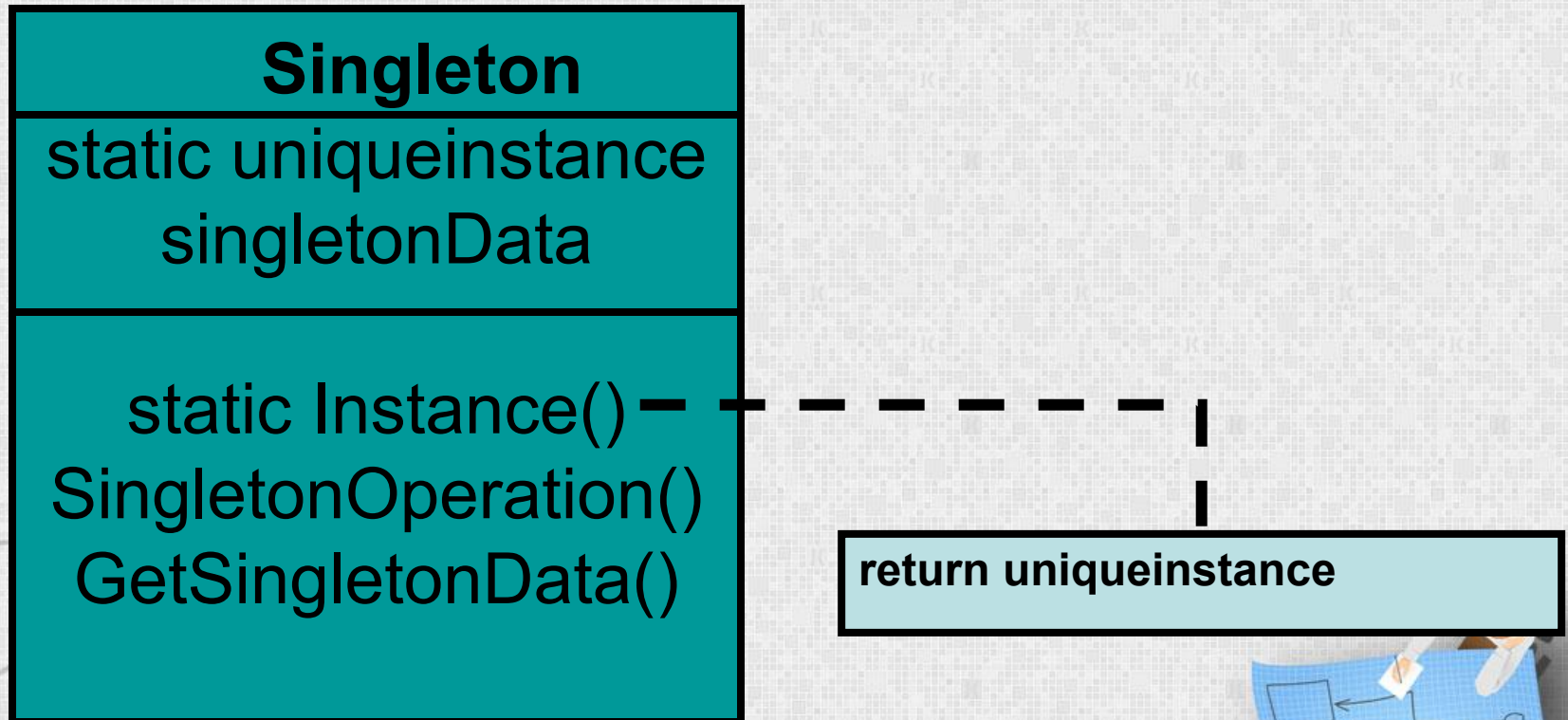
Singleton Pattern

Use the Singleton pattern when

- There must be exactly one instance of a class, and it must be accessible to clients from a well-known access point.



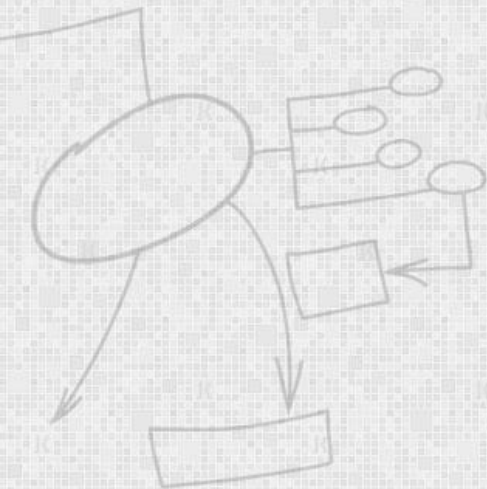
Singleton Structure



Singleton Implementation

- Ensuring a unique instance

A common way to do this is to hide the operation that creates the instance behind a static class operation that guarantees that only one instance is created.

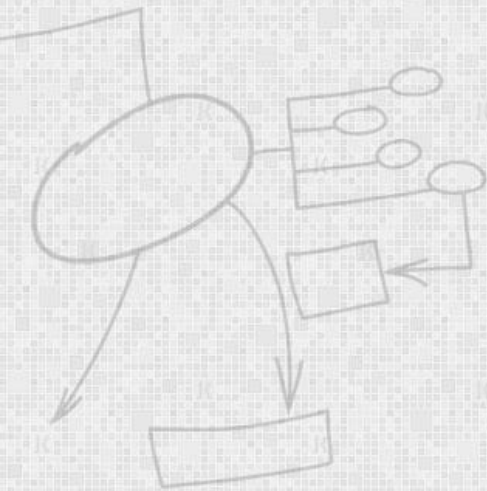


Singleton Implementation

```
class classteacher
{
    private string name;
    private static classteacher obj;
    private classteacher()
    { }
    public static classteacher Obj(string name)
    {
        if (obj == null)
        {
            obj = new classteacher();
            obj.name = name;
        }
        return obj;
    }
    public string display()
    {
        return name;
    }
}
```

Façade Design Pattern

Provide a unified interface to a set of interfaces in a subsystem.
Defines a high level interface that makes the subsystem easier to use.



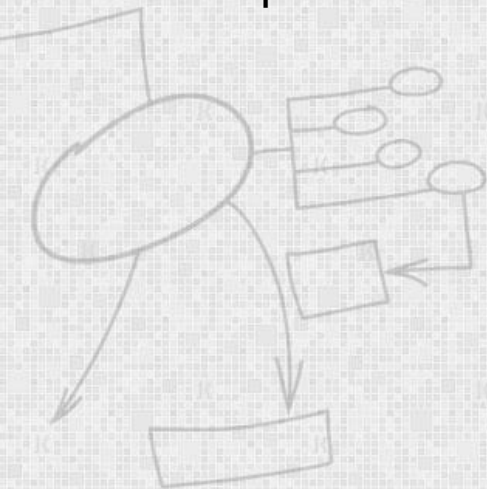
Facade

✓ Problem:

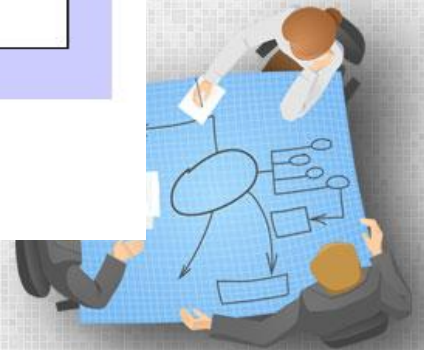
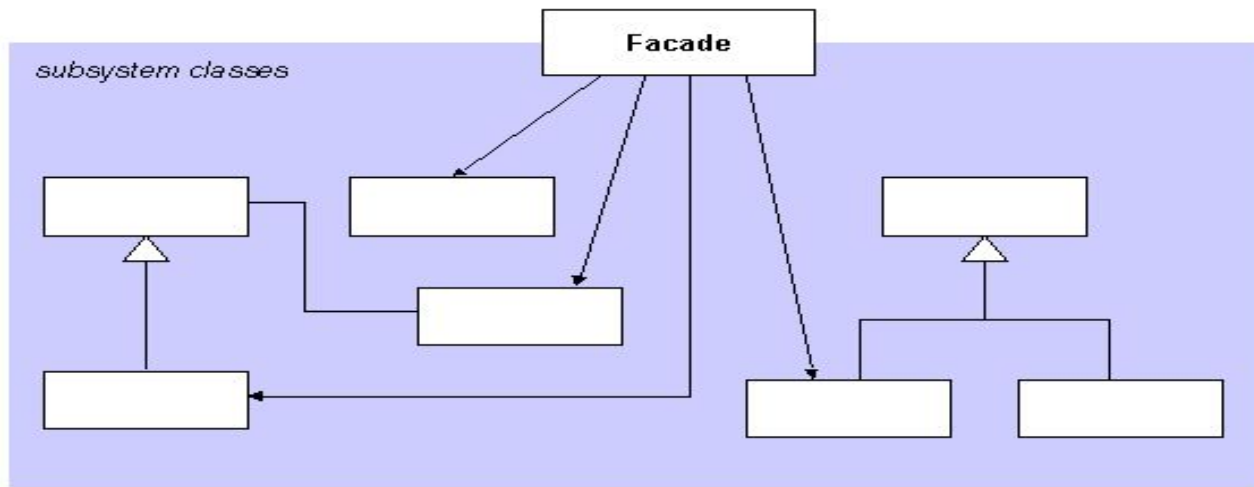
Need for a simplified interface to the overall functionality of a complex system.

✓ Motivation:

Reduce Complexity: Introduce a façade object that provides a single, simplified interface to a more general facilities of a subsystem.



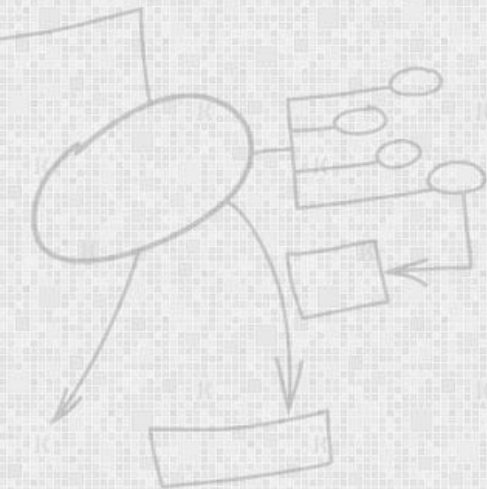
Facade - Structure

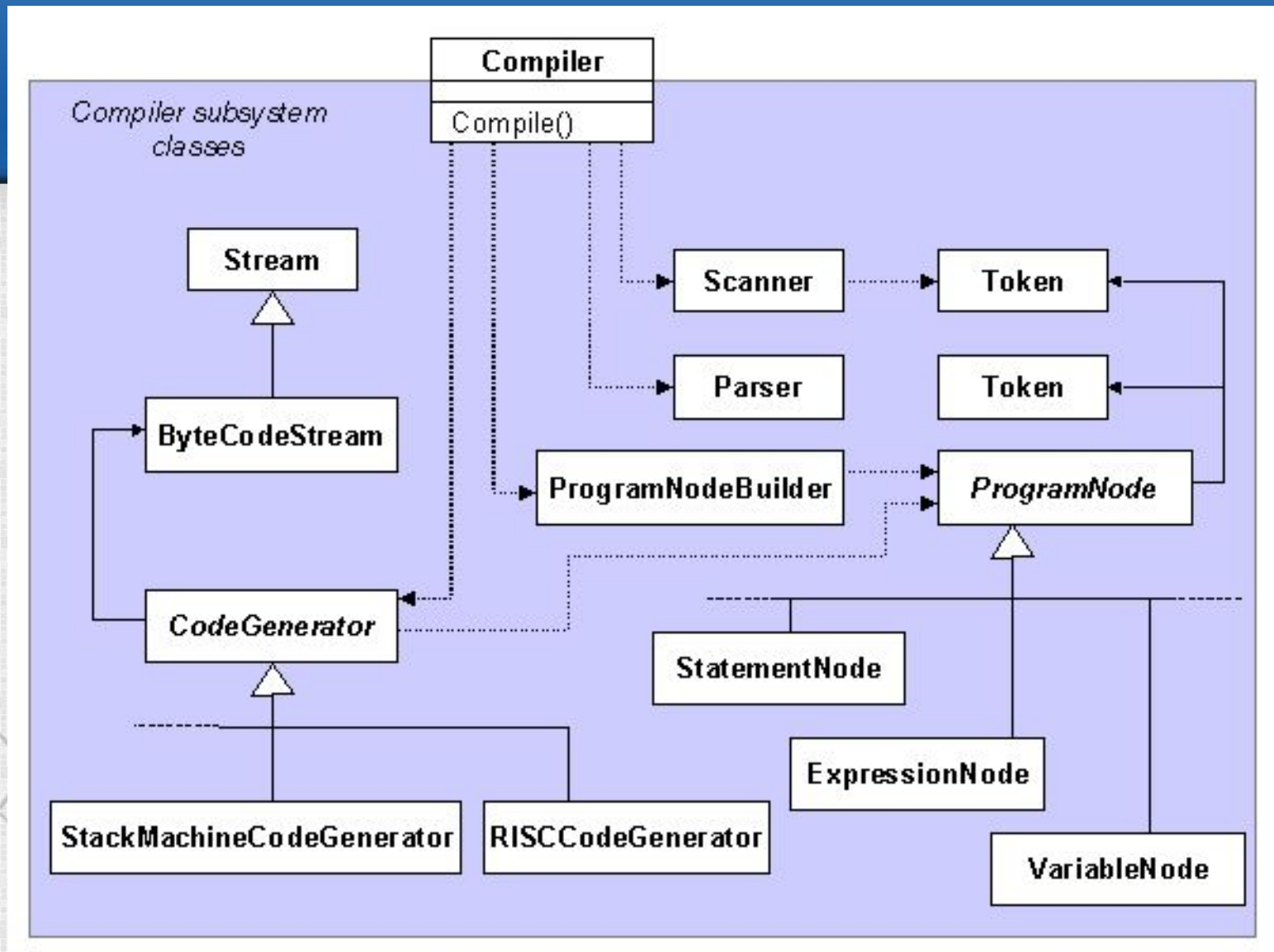


Facade - Participants

✓ Façade

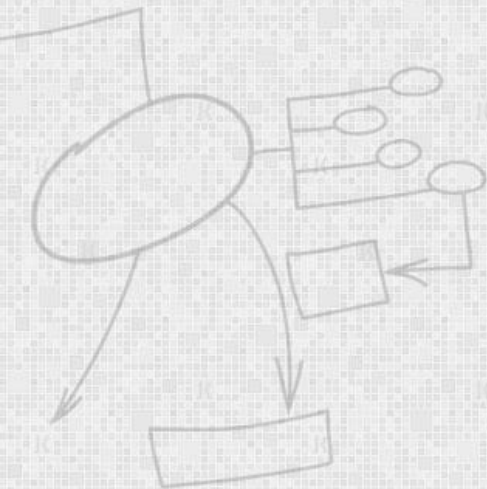
- Implements subsystem functionality
- Knows which subsystem classes are responsible for a request.
- Delegates client requests to appropriate subsystem objects





Facade – Known Users

- ✓ The compiler example was an ObjectWorks/SmallTalk compiler system
- ✓ Some operating systems are said to be using facades to compose many frameworks into one.

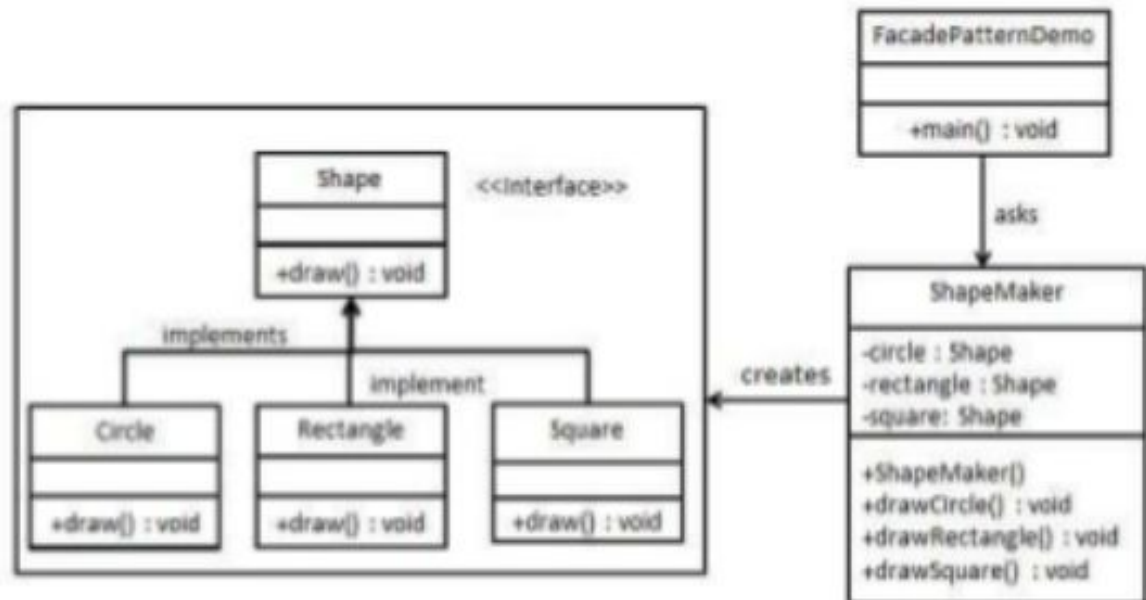


Facade

Implementation (C# and Java)

- We are going to implement following system class diagram.

Each time we need to create a 'Shape', we just interact with ShapeMaker (Façade) and it will manage the internal complexities.



Facade

Step 1

- Create an interface to enforce common method.
In our example it would be Shape interface containing a method draw.

Shape.java

```
public interface Shape {  
    void draw();  
}
```



Facade

Expenses

Step 2

- Define concrete classes and implement interface.

Rectangle.java

```
public class Rectangle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Rectangle::draw()");  
    }  
}
```

Square.java

```
public class Square implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Square::draw()");  
    }  
}
```

Circle.java

```
public class Circle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Circle::draw()");  
    }  
}
```



Facade

Step 3

Façade class acts as a wrapper class which contains all sub systems. In our example it will contain Circle, Rectangle and Square type of objects. Then you can have different necessary methods.

ShapeMaker.java

```
public class ShapeMaker {  
    private Shape circle;  
    private Shape rectangle;  
    private Shape square;  
  
    public ShapeMaker() {  
        circle = new Circle();  
        rectangle = new Rectangle();  
        square = new Square();  
    }  
  
    public void drawCircle(){  
        circle.draw();  
    }  
    public void drawRectangle(){  
        rectangle.draw();  
    }  
    public void drawSquare(){  
        square.draw();  
    }  
}
```

Facade

Step 4

- Use Façade class to use sub systems (to draw different shapes).

FacadePatternDemo.java

```
public class FacadePatternDemo {  
    public static void main(String[] args) {  
        ShapeMaker shapeMaker = new ShapeMaker();  
  
        shapeMaker.drawCircle();  
        shapeMaker.drawRectangle();  
        shapeMaker.drawSquare();  
    }  
}
```